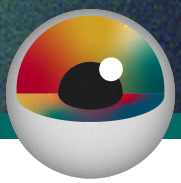


CORESENSE

ROS 2 Lifecycle Nodes

A Complete Tutorial with Python & C++ Examples
Covers: Humble / Iron / Jazzy distributions

Claude

v 1.0 — May 5, 2026

Abstract: This document is a self-contained tutorial on ROS 2 *Lifecycle Nodes*. It includes full code examples in both Python and C++, build-system files, command-line usage, and a hands-on walkthrough. Appendix A covers installation of ROS 2 and workspace preparation on Ubuntu Linux.

Contents

1 Introduction

1.1 What is a Lifecycle Node?

In standard ROS 2, a node starts executing immediately upon creation, and the only way to stop it is to kill the process. This makes it difficult to coordinate the startup and shutdown of complex multi-node systems.

Lifecycle Nodes (also called *managed nodes*) address this by giving each node a well-defined state machine. A lifecycle node does not begin meaningful work until it is explicitly told to do so by an external manager or by user commands. The operator can also pause, reconfigure, and cleanly shut down the node at run-time without restarting the whole system.

Lifecycle nodes are standardised in REP 2006 and implemented in the **`rclcpp_lifecycle`** (C++) and **`rclpy`** (Python, via `LifecycleNode`) packages.

1.2 Why Use Lifecycle Nodes?

- **Deterministic start-up**: ensure all dependencies are ready before a node becomes active.
- **Safe configuration**: allocate hardware resources only when needed; release them cleanly on shutdown.
- **Fault isolation**: deactivate a misbehaving node without restarting the entire robot pipeline.
- **System orchestration**: a *lifecycle manager* (e.g. from **`nav2_lifecycle_manager`**) can supervise multiple nodes as a unit.
- **Better testing**: each state transition can be unit-tested independently.

1.3 Scope of this Tutorial

1. The lifecycle state machine in detail.
2. Writing lifecycle nodes in **Python** and **C++**.
3. Building with `colcon` and `CMake`/`setuptools`.
4. Triggering transitions from the command line and programmatically.
5. Monitoring state via services and topics.
6. Implementing a simple lifecycle manager.
7. Best practices and common pitfalls.
8. Appendix: installing ROS 2 and setting up a workspace on Ubuntu.

2 The Lifecycle State Machine

2.1 Primary States

A lifecycle node always resides in one of the following *primary states*:

State	Description
`Unconfigured`	Node has been created but not yet configured. No publishers, subscribers, or timers are active.
`Inactive`	Node has been configured (resources allocated) but is not processing data.
`Active`	Node is fully operational and processing data.
`Finalized`	Node has been shut down and cannot be restarted.

2.2 Transition States

Between primary states the node briefly enters a *transition state* while executing callback logic. If the callback succeeds the node moves to the target primary state; if it fails the node moves to an error state instead.

Transition	From	To (success)	To (failure)
`configure`	Unconfigured	Inactive	Unconfigured
`activate`	Inactive	Active	Inactive
`deactivate`	Active	Inactive	Active
`cleanup`	Inactive	Unconfigured	Inactive
`shutdown`	Any primary	Finalized	Finalized
`error`	Any	Unconfigured	Finalized

2.3 Transition Callbacks

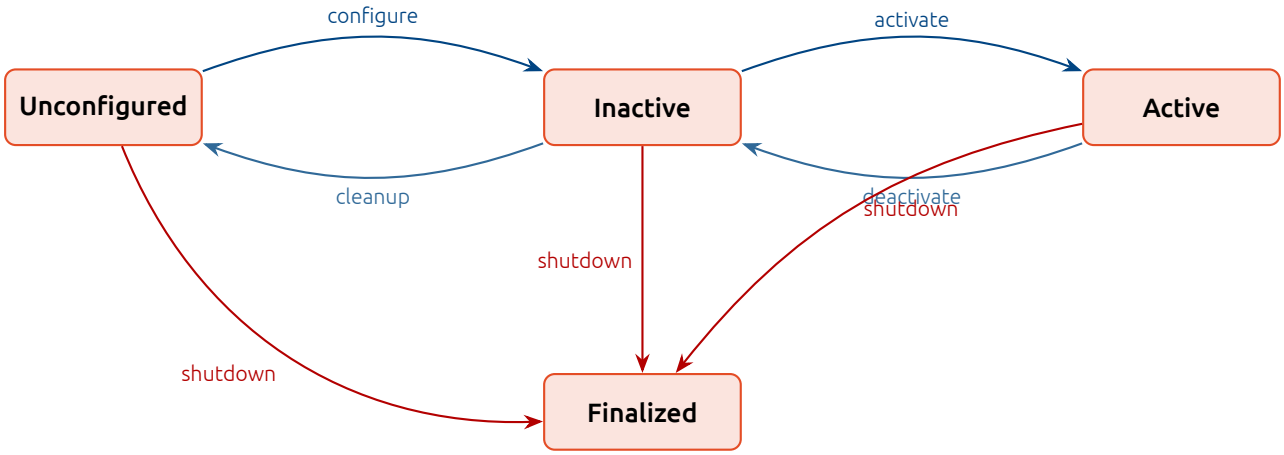
Each transition maps to a callback that you override in your node class:

Callback	Triggered by
`on_configure`	`configure` transition
`on_activate`	`activate` transition
`on_deactivate`	`deactivate` transition
`on_cleanup`	`cleanup` transition
`on_shutdown`	`shutdown` transition
`on_error`	Error recovery

All callbacks must return a *transition result*:

- **Python**: return `TransitionCallbackReturn.SUCCESS`, `.FAILURE`, or `.ERROR`.
- **C++**: return `CallbackReturn::SUCCESS`, `CallbackReturn::FAILURE`, or `CallbackReturn::ERROR`.

2.4 Full State-Machine Diagram



3 Lifecycle Nodes in Python

3.1 Package Structure

```
my_lifecycle_py/
  my_lifecycle_py/
    __init__.py
    lifecycle_talker.py       # publisher lifecycle node
    lifecycle_listener.py     # subscriber lifecycle node
    lifecycle_manager.py      # simple manager node
  resource/
    my_lifecycle_py
  package.xml
  setup.cfg
  setup.py
```

Listing 1: *Python package layout*

3.2 package.xml

```
<?xml version="1.0"?>
<?xml-model href="http://download.ros.org/schema/package_format3.xsd
    "
            schematypens="http://www.w3.org/2001/XMLSchema"?>
<package format="3">
  <name>my_lifecycle_py</name>
  <version>0.1.0</version>
  <description>ROS 2 lifecycle node examples in Python</description>
  <maintainer email="dev@example.com">Developer</maintainer>
  <license>Apache-2.0</license>

  <exec_depend>rclpy</exec_depend>
  <exec_depend>lifecycle_msgs</exec_depend>
  <exec_depend>std_msgs</exec_depend>

  <test_depend>ament_copyright</test_depend>
  <test_depend>ament_flake8</test_depend>
```

```
  <test_depend>ament_pep257</test_depend>

  <export>
    <build_type>ament_python</build_type>
  </export>
</package>
```

Listing 2: *package.xml for the Python package*

3.3 setup.py

```
from setuptools import setup

package_name = 'my_lifecycle_py'

setup(
    name=package_name,
    version='0.1.0',
    packages=[package_name],
    data_files=[
        ('share/ament_index/resource_index/packages',
         ['resource/' + package_name]),
        ('share/' + package_name, ['package.xml']),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='Developer',
    maintainer_email='dev@example.com',
    description='ROS 2 lifecycle node examples in Python',
    license='Apache-2.0',
    entry_points={
        'console_scripts': [
            'lifecycle_talker   = my_lifecycle_py.lifecycle_talker:main',
            'lifecycle_listener = my_lifecycle_py.lifecycle_listener:main',
            'lifecycle_manager  = my_lifecycle_py.lifecycle_manager:main',
        ],
    },
)
```

Listing 3: *setup.py*

3.4 Python -- Lifecycle Talker (Publisher Node)

The following example implements a lifecycle node that publishes a `std_msgs/String` message only while in the *Active* state. It demonstrates the four main transition callbacks.

```
# Copyright 2024 Developer
# SPDX-License-Identifier: Apache-2.0
"""Lifecycle talker: publishes only while Active."""

```

```python
import rclpy
from rclpy.lifecycle import LifecycleNode
from rclpy.lifecycle import TransitionCallbackReturn
from rclpy.lifecycle import State
from rclpy.timer import Timer
from std_msgs.msg import String


class LifecycleTalker(LifecycleNode):
    """A lifecycle node that publishes 'Hello' messages."""

    def __init__(self, node_name: str, **kwargs) -> None:
        """Initialise without creating ROS entities yet."""
        super().__init__(node_name, **kwargs)
        self._pub = None
        self._timer: Timer | None = None
        self._count: int = 0
        self.get_logger().info('LifecycleTalker created (Unconfigured)')

    # -- Transition callbacks ------------------------------------------

    def on_configure(self, state: State) -> TransitionCallbackReturn:
        """Allocate resources: create publisher.

        Called by the 'configure' transition.
        Returning SUCCESS moves the node to Inactive.
        """
        self.get_logger().info(
            f'Configuring from state: {state.label}')

        # Create publisher -- it exists but is NOT active yet
        self._pub = self.create_lifecycle_publisher(
            String, 'lifecycle_chatter', 10)

        self._count = 0
        self.get_logger().info('Publisher created.')
        return TransitionCallbackReturn.SUCCESS

    def on_activate(self, state: State) -> TransitionCallbackReturn:
        """Start publishing: create a timer that fires every second.

        Called by the 'activate' transition.
        Returning SUCCESS moves the node to Active.
        """
        self.get_logger().info(
            f'Activating from state: {state.label}')

        # Timer is created here so it fires only in Active state
        self._timer = self.create_timer(1.0, self._timer_callback)

        # IMPORTANT: call super().on_activate() to activate the
```

```
    publisher
        return super().on_activate(state)

    def on_deactivate(self, state: State) ->
    TransitionCallbackReturn:
        """Pause publishing: destroy the timer.

        Called by the 'deactivate' transition.
        Returning SUCCESS moves the node to Inactive.
        """
        self.get_logger().info(
            f'Deactivating from state: {state.label}')

        if self._timer:
            self._timer.destroy()
            self._timer = None

        # Call super to deactivate the publisher
        return super().on_deactivate(state)

    def on_cleanup(self, state: State) -> TransitionCallbackReturn:
        """Release all resources.

        Called by the 'cleanup' transition.
        Returning SUCCESS moves the node back to Unconfigured.
        """
        self.get_logger().info(
            f'Cleaning up from state: {state.label}')

        if self._timer:
            self._timer.destroy()
            self._timer = None

        if self._pub:
            self.destroy_publisher(self._pub)
            self._pub = None

        self._count = 0
        self.get_logger().info('Resources released.')
        return TransitionCallbackReturn.SUCCESS

    def on_shutdown(self, state: State) -> TransitionCallbackReturn:
        """Graceful shutdown -- called from any primary state."""
        self.get_logger().info(
            f'Shutting down from state: {state.label}')

        # Re-use cleanup logic
        if self._timer:
            self._timer.destroy()
            self._timer = None
        if self._pub:
            self.destroy_publisher(self._pub)
            self._pub = None

```

```
        return TransitionCallbackReturn.SUCCESS

    def on_error(self, state: State) -> TransitionCallbackReturn:
        """Error recovery handler."""
        self.get_logger().error(
            f'Error in state: {state.label}. Attempting recovery.')
        # Attempt to clean up; returning FAILURE sends to Finalized
        return TransitionCallbackReturn.SUCCESS

    # -- Internal
    ------------------------------------------------------------

    def _timer_callback(self) -> None:
        """Publish a message. Only called while Active."""
        if self._pub and self._pub.is_activated:
            msg = String()
            msg.data = f'Hello, lifecycle world! count={self._count}
    '
            self._pub.publish(msg)
            self.get_logger().info(f'Published: "{msg.data}"')
            self._count += 1


def main(args=None):
    rclpy.init(args=args)
    node = LifecycleTalker('lifecycle_talker')
    executor = rclpy.executors.SingleThreadedExecutor()
    executor.add_node(node)
    try:
        executor.spin()
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()


if __name__ == '__main__':
    main()
```

Listing 4: *lifecycle_talker.py – full example*

Note

Key point: use `create_lifecycle_publisher()` instead of `create_publisher()`. A lifecycle publisher is automatically *activated* and *deactivated* with the node, preventing messages from being published in the wrong state. Always call `super().on_activate(state)` and `super().on_deactivate(state)` to trigger this behaviour.

3.5 Python -- Lifecycle Listener (Subscriber Node)

```
# Copyright 2024 Developer
# SPDX-License-Identifier: Apache-2.0
"""Lifecycle listener: receives messages only while Active."""
```

```python

import rclpy
from rclpy.lifecycle import LifecycleNode
from rclpy.lifecycle import TransitionCallbackReturn
from rclpy.lifecycle import State
from std_msgs.msg import String


class LifecycleListener(LifecycleNode):
    """A lifecycle node that subscribes to the chatter topic."""

    def __init__(self, node_name: str, **kwargs) -> None:
        super().__init__(node_name, **kwargs)
        self._sub = None
        self.get_logger().info('LifecycleListener created (Unconfigured)')

    def on_configure(self, state: State) -> TransitionCallbackReturn:
        self.get_logger().info('Configuring listener...')
        # Subscription is created here; messages are queued but the
        # callback fires only when the node is Active.
        self._sub = self.create_subscription(
            String,
            'lifecycle_chatter',
            self._chatter_callback,
            10,
        )
        return TransitionCallbackReturn.SUCCESS

    def on_activate(self, state: State) -> TransitionCallbackReturn:
        self.get_logger().info('Activating listener...')
        return super().on_activate(state)

    def on_deactivate(self, state: State) -> TransitionCallbackReturn:
        self.get_logger().info('Deactivating listener...')
        return super().on_deactivate(state)

    def on_cleanup(self, state: State) -> TransitionCallbackReturn:
        self.get_logger().info('Cleaning up listener...')
        if self._sub:
            self.destroy_subscription(self._sub)
            self._sub = None
        return TransitionCallbackReturn.SUCCESS

    def on_shutdown(self, state: State) -> TransitionCallbackReturn:
        self.get_logger().info('Shutting down listener...')
        if self._sub:
            self.destroy_subscription(self._sub)
            self._sub = None
        return TransitionCallbackReturn.SUCCESS

    def _chatter_callback(self, msg: String) -> None:
```

```
64          self.get_logger().info(f'Received: "{msg.data}"')
65
66
67  def main(args=None):
68      rclpy.init(args=args)
69      node = LifecycleListener('lifecycle_listener')
70      executor = rclpy.executors.SingleThreadedExecutor()
71      executor.add_node(node)
72      try:
73          executor.spin()
74      except KeyboardInterrupt:
75          pass
76      finally:
77          node.destroy_node()
78          rclpy.shutdown()
79
80
81  if __name__ == '__main__':
82      main()
```

Listing 5: *lifecycle_listener.py*

3.6 Python -- Programmatic Lifecycle Manager

The following node drives another lifecycle node through its states programmatically by calling lifecycle change-state services.

```
10  # Copyright 2024 Developer
11  # SPDX-License-Identifier: Apache-2.0
12  """Simple lifecycle manager that drives another node through its
       states."""
13
14  import time
15  import rclpy
16  from rclpy.node import Node
17  from lifecycle_msgs.srv import ChangeState, GetState
18  from lifecycle_msgs.msg import Transition
19
20
21  TRANSITIONS = {
22      'configure':  Transition.TRANSITION_CONFIGURE,
23      'activate':   Transition.TRANSITION_ACTIVATE,
24      'deactivate': Transition.TRANSITION_DEACTIVATE,
25      'cleanup':    Transition.TRANSITION_CLEANUP,
26      'shutdown':   Transition.TRANSITION_UNCONFIGURED_SHUTDOWN,
27  }
28
29
30  class LifecycleManager(Node):
31      """Drives a target lifecycle node through a predefined sequence.
       """
32
33       def __init__(self, target_node: str = 'lifecycle_talker') ->
       None:
```

```
        super().__init__('lifecycle_manager')
        self._target = target_node

        # Create service clients
        self._change_state = self.create_client(
            ChangeState,
            f'/{target_node}/change_state',
        )
        self._get_state = self.create_client(
            GetState,
            f'/{target_node}/get_state',
        )
        self.get_logger().info(
            f'LifecycleManager ready to manage: {target_node}')

    # -- Public API
    # ------------------------------------------------------------

    def get_state(self) -> str | None:
        """Return the current primary state label of the managed node."""
        if not self._get_state.wait_for_service(timeout_sec=5.0):
            self.get_logger().error('get_state service not available')
            return None
        future = self._get_state.call_async(GetState.Request())
        rclpy.spin_until_future_complete(self, future, timeout_sec=5.0)
        if future.result():
            return future.result().current_state.label
        return None

    def change_state(self, transition_name: str) -> bool:
        """Trigger a named transition. Returns True on success."""
        if transition_name not in TRANSITIONS:
            self.get_logger().error(
                f'Unknown transition: {transition_name}')
            return False

        if not self._change_state.wait_for_service(timeout_sec=5.0):
            self.get_logger().error('change_state service not available')
            return False

        req = ChangeState.Request()
        req.transition.id = TRANSITIONS[transition_name]
        req.transition.label = transition_name

        future = self._change_state.call_async(req)
        rclpy.spin_until_future_complete(self, future, timeout_sec=10.0)

        if future.result() and future.result().success:
            state = self.get_state()
```

```
            self.get_logger().info(
                f'Transition "{transition_name}" succeeded. '
                f'New state: {state}')
            return True
        else:
            self.get_logger().error(
                f'Transition "{transition_name}" FAILED.')
            return False

    def run_demo_sequence(self) -> None:
        """Run configure -> activate -> (wait) -> deactivate -> cleanup."""
        steps = [
            ('configure',  2.0),
            ('activate',   5.0),
            ('deactivate', 1.0),
            ('cleanup',    1.0),
        ]
        for transition, wait in steps:
            if not self.change_state(transition):
                self.get_logger().error(
                    f'Aborting sequence at: {transition}')
                return
            time.sleep(wait)

        self.get_logger().info('Demo sequence complete.')


def main(args=None):
    rclpy.init(args=args)
    manager = LifecycleManager(target_node='lifecycle_talker')

    # Wait briefly for the talker to come up
    time.sleep(2.0)
    manager.run_demo_sequence()

    manager.destroy_node()
    rclpy.shutdown()


if __name__ == '__main__':
    main()
```

Listing 6: *lifecycle_manager.py*

4 Lifecycle Nodes in C++

4.1 Package Structure

```
my_lifecycle_cpp/
  include/
    my_lifecycle_cpp/
      lifecycle_talker.hpp
```

```
      lifecycle_listener.hpp
      lifecycle_manager.hpp
  src/
    lifecycle_talker.cpp
    lifecycle_listener.cpp
    lifecycle_manager.cpp
  package.xml
  CMakeLists.txt
```

Listing 7: *C++ package layout*

4.2 package.xml

```xml
<?xml version="1.0"?>
<package format="3">
  <name>my_lifecycle_cpp</name>
  <version>0.1.0</version>
  <description>ROS 2 lifecycle node examples in C++</description>
  <maintainer email="dev@example.com">Developer</maintainer>
  <license>Apache-2.0</license>

  <buildtool_depend>ament_cmake</buildtool_depend>

  <depend>rclcpp</depend>
  <depend>rclcpp_lifecycle</depend>
  <depend>lifecycle_msgs</depend>
  <depend>std_msgs</depend>

  <test_depend>ament_lint_auto</test_depend>
  <test_depend>ament_lint_common</test_depend>

  <export>
    <build_type>ament_cmake</build_type>
  </export>
</package>
```

Listing 8: *package.xml for the C++ package*

4.3 CMakeLists.txt

```
cmake_minimum_required(VERSION 3.14)
project(my_lifecycle_cpp)

# C++17 required for std::optional, structured bindings, etc.
if(NOT CMAKE_CXX_STANDARD)
  set(CMAKE_CXX_STANDARD 17)
endif()
if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES "Clang"
   )
  add_compile_options(-Wall -Wextra -Wpedantic)
endif()

# Find dependencies
```

```
find_package(ament_cmake REQUIRED)
find_package(rclcpp REQUIRED)
find_package(rclcpp_lifecycle REQUIRED)
find_package(lifecycle_msgs REQUIRED)
find_package(std_msgs REQUIRED)

set(DEPS
  rclcpp rclcpp_lifecycle lifecycle_msgs std_msgs)

# -- Lifecycle Talker
  ------------------------------------------------------------
add_executable(lifecycle_talker src/lifecycle_talker.cpp)
ament_target_dependencies(lifecycle_talker ${DEPS})
target_include_directories(lifecycle_talker PUBLIC
  $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include>)

# -- Lifecycle Listener
  ------------------------------------------------------------
add_executable(lifecycle_listener src/lifecycle_listener.cpp)
ament_target_dependencies(lifecycle_listener ${DEPS})
target_include_directories(lifecycle_listener PUBLIC
  $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include>)

# -- Lifecycle Manager
  ------------------------------------------------------------
add_executable(lifecycle_manager src/lifecycle_manager.cpp)
ament_target_dependencies(lifecycle_manager ${DEPS})
target_include_directories(lifecycle_manager PUBLIC
  $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include>)

# Install
install(TARGETS
  lifecycle_talker lifecycle_listener lifecycle_manager
  DESTINATION lib/${PROJECT_NAME})

if(BUILD_TESTING)
  find_package(ament_lint_auto REQUIRED)
  ament_lint_auto_find_test_dependencies()
endif()

ament_package()
```

Listing 9: *CMakeLists.txt*

4.4 C++ -- Lifecycle Talker Header

```
// Copyright 2024 Developer
// SPDX-License-Identifier: Apache-2.0
#ifndef MY_LIFECYCLE_CPP__LIFECYCLE_TALKER_HPP_
#define MY_LIFECYCLE_CPP__LIFECYCLE_TALKER_HPP_

#include <memory>
#include <string>

```

```cpp
#include "rclcpp/rclcpp.hpp"
#include "rclcpp_lifecycle/lifecycle_node.hpp"
#include "rclcpp_lifecycle/lifecycle_publisher.hpp"
#include "std_msgs/msg/string.hpp"

namespace my_lifecycle_cpp
{

using CallbackReturn =
  rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::
   CallbackReturn;

/**
 * @brief Lifecycle talker node.
 *
 * Publishes std_msgs::msg::String messages at 1 Hz only while
   Active.
 */
class LifecycleTalker : public rclcpp_lifecycle::LifecycleNode
{
public:
  /**
   * @brief Construct the talker.
   * @param options  Node options forwarded to LifecycleNode.
   */
  explicit LifecycleTalker(
    const rclcpp::NodeOptions & options = rclcpp::NodeOptions());

  ~LifecycleTalker() override = default;

  // -- Transition callbacks
   ------------------------------------------------
  CallbackReturn on_configure(const rclcpp_lifecycle::State & state)
    override;
  CallbackReturn on_activate  (const rclcpp_lifecycle::State & state
   ) override;
  CallbackReturn on_deactivate(const rclcpp_lifecycle::State & state
   ) override;
  CallbackReturn on_cleanup   (const rclcpp_lifecycle::State & state
   ) override;
  CallbackReturn on_shutdown  (const rclcpp_lifecycle::State & state
   ) override;

private:
  void timer_callback();

  rclcpp_lifecycle::LifecyclePublisher<std_msgs::msg::String>::
   SharedPtr pub_;
  rclcpp::TimerBase::SharedPtr timer_;
  size_t count_{0};
};

}  // namespace my_lifecycle_cpp
#endif  // MY_LIFECYCLE_CPP__LIFECYCLE_TALKER_HPP_
```

Listing 10: *include/my_lifecycle_cpp/lifecycle_talker.hpp*

4.5 C++ -- Lifecycle Talker Implementation

```cpp
// Copyright 2024 Developer
// SPDX-License-Identifier: Apache-2.0
#include "my_lifecycle_cpp/lifecycle_talker.hpp"

#include <chrono>
#include <memory>
#include <string>

using namespace std::chrono_literals;

namespace my_lifecycle_cpp
{

LifecycleTalker::LifecycleTalker(const rclcpp::NodeOptions & options)
: rclcpp_lifecycle::LifecycleNode("lifecycle_talker", options)
{
  RCLCPP_INFO(get_logger(), "LifecycleTalker created (Unconfigured)");
}

// -- on_configure ------------------------------------------------------------
CallbackReturn
LifecycleTalker::on_configure(const rclcpp_lifecycle::State & state)
{
  RCLCPP_INFO(get_logger(),
    "Configuring from state: %s", state.label().c_str());

  // Create a lifecycle publisher.
  // It is inactive until on_activate() is called.
  pub_ = create_lifecycle_publisher<std_msgs::msg::String>(
    "lifecycle_chatter", 10);

  count_ = 0;
  RCLCPP_INFO(get_logger(), "Publisher created.");
  return CallbackReturn::SUCCESS;
}

// -- on_activate -------------------------------------------------------------
CallbackReturn
LifecycleTalker::on_activate(const rclcpp_lifecycle::State & state)
{
  RCLCPP_INFO(get_logger(),
    "Activating from state: %s", state.label().c_str());

  // Create the timer that triggers publishing
```

```
54    timer_ = create_wall_timer(
55      1s, std::bind(&LifecycleTalker::timer_callback, this));
56
57    // IMPORTANT: call the parent implementation to activate the
       publisher
58    LifecycleNode::on_activate(state);
59    return CallbackReturn::SUCCESS;
60  }
61
62  // -- on_deactivate
       ---------------------------------------------------------------
63  CallbackReturn
64  LifecycleTalker::on_deactivate(const rclcpp_lifecycle::State & state
       )
65  {
66    RCLCPP_INFO(get_logger(),
67      "Deactivating from state: %s", state.label().c_str());
68
69    // Destroy the timer to stop publishing
70    timer_.reset();
71
72    // Deactivate the publisher through the parent
73    LifecycleNode::on_deactivate(state);
74    return CallbackReturn::SUCCESS;
75  }
76
77  // -- on_cleanup
       ---------------------------------------------------------------
78  CallbackReturn
79  LifecycleTalker::on_cleanup(const rclcpp_lifecycle::State & state)
80  {
81    RCLCPP_INFO(get_logger(),
82      "Cleaning up from state: %s", state.label().c_str());
83
84    timer_.reset();
85    pub_.reset();
86    count_ = 0;
87    RCLCPP_INFO(get_logger(), "Resources released.");
88    return CallbackReturn::SUCCESS;
89  }
90
91  // -- on_shutdown
       ---------------------------------------------------------------
92  CallbackReturn
93  LifecycleTalker::on_shutdown(const rclcpp_lifecycle::State & state)
94  {
95    RCLCPP_INFO(get_logger(),
96      "Shutting down from state: %s", state.label().c_str());
97
98    timer_.reset();
99    pub_.reset();
100   return CallbackReturn::SUCCESS;
101 }
102
```

```cpp
// -- timer_callback
   ------------------------------------------------------------------
void LifecycleTalker::timer_callback()
{
  if (!pub_ || !pub_->is_activated()) {
    return;
  }
  auto msg = std_msgs::msg::String();
  msg.data = "Hello, lifecycle world! count=" + std::to_string(
   count_++);
  pub_->publish(msg);
  RCLCPP_INFO(get_logger(), "Published: '%s'", msg.data.c_str());
}

}  // namespace my_lifecycle_cpp

// -- main
   ------------------------------------------------------------------------

int main(int argc, char ** argv)
{
  rclcpp::init(argc, argv);
  auto node = std::make_shared<my_lifecycle_cpp::LifecycleTalker>();
  rclcpp::spin(node->get_node_base_interface());
  rclcpp::shutdown();
  return 0;
}
```

Listing 11: *src/lifecycle_talker.cpp*

4.6 C++ -- Lifecycle Listener Header

```cpp
// Copyright 2024 Developer
// SPDX-License-Identifier: Apache-2.0
#ifndef MY_LIFECYCLE_CPP__LIFECYCLE_LISTENER_HPP_
#define MY_LIFECYCLE_CPP__LIFECYCLE_LISTENER_HPP_

#include <memory>

#include "rclcpp/rclcpp.hpp"
#include "rclcpp_lifecycle/lifecycle_node.hpp"
#include "std_msgs/msg/string.hpp"

namespace my_lifecycle_cpp
{

using CallbackReturn =
  rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::
   CallbackReturn;

class LifecycleListener : public rclcpp_lifecycle::LifecycleNode
{
public:
  explicit LifecycleListener(
```

```
31      const rclcpp::NodeOptions & options = rclcpp::NodeOptions());
32
33    CallbackReturn on_configure (const rclcpp_lifecycle::State &)
       override;
34    CallbackReturn on_activate  (const rclcpp_lifecycle::State &)
       override;
35    CallbackReturn on_deactivate(const rclcpp_lifecycle::State &)
       override;
36    CallbackReturn on_cleanup   (const rclcpp_lifecycle::State &)
       override;
37    CallbackReturn on_shutdown  (const rclcpp_lifecycle::State &)
       override;
38
39  private:
40    void chatter_callback(const std_msgs::msg::String & msg);
41    rclcpp::Subscription<std_msgs::msg::String>::SharedPtr sub_;
42  };
43
44  }  // namespace my_lifecycle_cpp
45  #endif  // MY_LIFECYCLE_CPP__LIFECYCLE_LISTENER_HPP_
```

Listing 12: *include/my_lifecycle_cpp/lifecycle_listener.hpp*

4.7 C++ -- Lifecycle Listener Implementation

```
10  // Copyright 2024 Developer
11  // SPDX-License-Identifier: Apache-2.0
12  #include "my_lifecycle_cpp/lifecycle_listener.hpp"
13
14  #include <memory>
15  #include <functional>
16
17  namespace my_lifecycle_cpp
18  {
19
20  LifecycleListener::LifecycleListener(const rclcpp::NodeOptions &
       opts)
21  : rclcpp_lifecycle::LifecycleNode("lifecycle_listener", opts)
22  {
23    RCLCPP_INFO(get_logger(), "LifecycleListener created (Unconfigured
       )");
24  }
25
26  CallbackReturn
27  LifecycleListener::on_configure(const rclcpp_lifecycle::State &
       state)
28  {
29    RCLCPP_INFO(get_logger(), "Configuring: %s", state.label().c_str()
       );
30    sub_ = create_subscription<std_msgs::msg::String>(
31      "lifecycle_chatter", 10,
32      std::bind(&LifecycleListener::chatter_callback, this,
33                std::placeholders::_1));
34    return CallbackReturn::SUCCESS;
```

```
35 }
36
37 CallbackReturn
38 LifecycleListener::on_activate(const rclcpp_lifecycle::State & state
       )
39 {
40   RCLCPP_INFO(get_logger(), "Activating: %s", state.label().c_str())
       ;
41   return LifecycleNode::on_activate(state);
42 }
43
44 CallbackReturn
45 LifecycleListener::on_deactivate(const rclcpp_lifecycle::State &
       state)
46 {
47   RCLCPP_INFO(get_logger(), "Deactivating: %s", state.label().c_str
       ());
48   return LifecycleNode::on_deactivate(state);
49 }
50
51 CallbackReturn
52 LifecycleListener::on_cleanup(const rclcpp_lifecycle::State & state)
53 {
54   RCLCPP_INFO(get_logger(), "Cleaning up: %s", state.label().c_str()
       );
55   sub_.reset();
56   return CallbackReturn::SUCCESS;
57 }
58
59 CallbackReturn
60 LifecycleListener::on_shutdown(const rclcpp_lifecycle::State & state
       )
61 {
62   RCLCPP_INFO(get_logger(), "Shutting down: %s", state.label().c_str
       ());
63   sub_.reset();
64   return CallbackReturn::SUCCESS;
65 }
66
67 void LifecycleListener::chatter_callback(const std_msgs::msg::String
       & msg)
68 {
69   RCLCPP_INFO(get_logger(), "Received: '%s'", msg.data.c_str());
70 }
71
72 }  // namespace my_lifecycle_cpp
73
74 int main(int argc, char ** argv)
75 {
76   rclcpp::init(argc, argv);
77   auto node = std::make_shared<my_lifecycle_cpp::LifecycleListener
       >();
78   rclcpp::spin(node->get_node_base_interface());
79   rclcpp::shutdown();
```

```
  return 0;
}
```

Listing 13: *src/lifecycle_listener.cpp*

4.8 C++ -- Lifecycle Manager

```
// Copyright 2024 Developer
// SPDX-License-Identifier: Apache-2.0
/**
 * @brief Simple lifecycle manager that orchestrates another lifecycle node.
 *
 * Calls /<target>/change_state and /<target>/get_state services.
 */
#include <chrono>
#include <memory>
#include <string>
#include <thread>

#include "rclcpp/rclcpp.hpp"
#include "lifecycle_msgs/srv/change_state.hpp"
#include "lifecycle_msgs/srv/get_state.hpp"
#include "lifecycle_msgs/msg/transition.hpp"

using namespace std::chrono_literals;
using ChangeState = lifecycle_msgs::srv::ChangeState;
using GetState    = lifecycle_msgs::srv::GetState;

class LifecycleManager : public rclcpp::Node
{
public:
  explicit LifecycleManager(const std::string & target = "lifecycle_talker")
  : Node("lifecycle_manager"), target_(target)
  {
    change_state_client_ = create_client<ChangeState>(
      "/" + target + "/change_state");
    get_state_client_ = create_client<GetState>(
      "/" + target + "/get_state");
    RCLCPP_INFO(get_logger(),
      "LifecycleManager ready to manage: %s", target_.c_str());
  }

  // -- get_state ------------------------------------------------------
  std::string get_state()
  {
    if (!get_state_client_->wait_for_service(5s)) {
      RCLCPP_ERROR(get_logger(), "get_state service unavailable");
      return "unknown";
    }
    auto req = std::make_shared<GetState::Request>();
    auto future = get_state_client_->async_send_request(req);
```

```
54      if (rclcpp::spin_until_future_complete(
55            get_node_base_interface(), future, 5s) ==
56          rclcpp::FutureReturnCode::SUCCESS)
57      {
58        return future.get()->current_state.label;
59      }
60      return "unknown";
61    }
62
63    // -- change_state
      ------------------------------------------------------------
64    bool change_state(uint8_t transition_id, const std::string & label
       )
65    {
66      if (!change_state_client_->wait_for_service(5s)) {
67        RCLCPP_ERROR(get_logger(), "change_state service unavailable")
       ;
68        return false;
69      }
70      auto req = std::make_shared<ChangeState::Request>();
71      req->transition.id    = transition_id;
72      req->transition.label = label;
73
74      auto future = change_state_client_->async_send_request(req);
75      if (rclcpp::spin_until_future_complete(
76            get_node_base_interface(), future, 10s) ==
77          rclcpp::FutureReturnCode::SUCCESS)
78      {
79        if (future.get()->success) {
80          RCLCPP_INFO(get_logger(),
81            "Transition '%s' succeeded. New state: %s",
82            label.c_str(), get_state().c_str());
83          return true;
84        }
85      }
86      RCLCPP_ERROR(get_logger(), "Transition '%s' FAILED.", label.
       c_str());
87      return false;
88    }
89
90    // -- convenience wrappers
      ------------------------------------------------------
91    bool configure()  {
92      return change_state(
93        lifecycle_msgs::msg::Transition::TRANSITION_CONFIGURE, "
       configure");
94    }
95    bool activate()   {
96      return change_state(
97        lifecycle_msgs::msg::Transition::TRANSITION_ACTIVATE, "
       activate");
98    }
99    bool deactivate() {
100     return change_state(
```

```
101       lifecycle_msgs::msg::Transition::TRANSITION_DEACTIVATE, "
    deactivate");
102   }
103   bool cleanup()    {
104     return change_state(
105       lifecycle_msgs::msg::Transition::TRANSITION_CLEANUP, "cleanup"
    );
106   }
107   bool shutdown()   {
108     return change_state(
109       lifecycle_msgs::msg::Transition::
110         TRANSITION_UNCONFIGURED_SHUTDOWN, "shutdown");
111   }
112
113   // -- demo sequence
      ------------------------------------------------------------
114   void run_demo()
115   {
116     std::this_thread::sleep_for(2s);  // wait for talker to start
117     if (!configure())  return;
118     std::this_thread::sleep_for(1s);
119     if (!activate())   return;
120     std::this_thread::sleep_for(5s);  // let it publish for 5 s
121     if (!deactivate()) return;
122     std::this_thread::sleep_for(1s);
123     if (!cleanup())    return;
124     RCLCPP_INFO(get_logger(), "Demo sequence complete.");
125   }
126
127 private:
128   std::string target_;
129   rclcpp::Client<ChangeState>::SharedPtr change_state_client_;
130   rclcpp::Client<GetState>::SharedPtr    get_state_client_;
131 };
132
133 int main(int argc, char ** argv)
134 {
135   rclcpp::init(argc, argv);
136   auto manager = std::make_shared<LifecycleManager>("
    lifecycle_talker");
137   manager->run_demo();
138   rclcpp::shutdown();
139   return 0;
140 }
```

Listing 14: *src/lifecycle_manager.cpp*

5 Building the Packages

5.1 Workspace Layout

Both packages should live inside a colcon workspace:

```
~/ros2_ws/
  src/
    my_lifecycle_py/    # Python package
    my_lifecycle_cpp/   # C++ package
```

Listing 15: *Workspace layout*

5.2 Building

```
# Source ROS 2 underlay (replace 'humble' with your distro)
source /opt/ros/humble/setup.bash

# Navigate to workspace root
cd ~/ros2_ws

# Build all packages
colcon build --symlink-install

# Source the overlay (so your packages are found)
source install/setup.bash
```

Listing 16: *Build commands*

Tip

`--symlink-install` links Python files directly from `src/` instead of copying them, so you can edit without rebuilding. This only applies to Python packages; C++ still needs a rebuild after source changes.

5.3 Building Individual Packages

```
# Build only the C++ package
colcon build --packages-select my_lifecycle_cpp

# Build only the Python package
colcon build --packages-select my_lifecycle_py --symlink-install
```

Listing 17: *Build a single package*

6 Running and Testing

6.1 Launching the Nodes

Open three terminals, each sourced with the workspace overlay.

Terminal 1 – start the talker (Python):

```
source ~/ros2_ws/install/setup.bash
ros2 run my_lifecycle_py lifecycle_talker
```

Terminal 2 – start the listener (Python):

```
source ~/ros2_ws/install/setup.bash
ros2 run my_lifecycle_py lifecycle_listener
```

Terminal 3 – trigger transitions manually:

6.2 Manual Transitions via CLI

The `ros2 lifecycle` CLI is the easiest way to explore the state machine.

```
# List available transitions for a node
ros2 lifecycle list /lifecycle_talker

# Get the current state
ros2 lifecycle get /lifecycle_talker

# Trigger the 'configure' transition
ros2 lifecycle set /lifecycle_talker configure

# Trigger 'activate'
ros2 lifecycle set /lifecycle_talker activate

# Watch messages being published
ros2 topic echo /lifecycle_chatter

# Deactivate -- publishing stops
ros2 lifecycle set /lifecycle_talker deactivate

# Clean up -- resources released
ros2 lifecycle set /lifecycle_talker cleanup

# Shut down
ros2 lifecycle set /lifecycle_talker shutdown
```

Listing 18: *CLI transition commands*

6.3 Using the Programmatic Manager

```
# In a separate terminal -- runs the full sequence automatically
ros2 run my_lifecycle_py lifecycle_manager

# Or the C++ version
ros2 run my_lifecycle_cpp lifecycle_manager
```

Listing 19: *Run the lifecycle manager*

6.4 Monitoring State via Topic

Lifecycle nodes automatically publish their current state on `/<node_name>/transition_event` as a `lifecycle_msgs/msg/TransitionEvent` message.

```
ros2 topic echo /lifecycle_talker/transition_event
```

Listing 20: *Monitor state transitions*

Sample output:

```
---
timestamp: 1711000000000000000
transition:
  id: 1
  label: configure
start_state:
  id: 1
  label: unconfigured
goal_state:
  id: 2
  label: inactive
---
```

7 Advanced Topics

7.1 Using Parameters Inside Lifecycle Nodes

Parameters are typically declared in `on_configure` so they are available for resource allocation:

```python
def on_configure(self, state: State) -> TransitionCallbackReturn:
    # Declare parameters with defaults
    self.declare_parameter('publish_rate', 1.0)
    self.declare_parameter('topic_name', 'lifecycle_chatter')
    self.declare_parameter('queue_size', 10)

    # Read them
    rate  = self.get_parameter('publish_rate').value
    topic = self.get_parameter('topic_name').value
    qsize = self.get_parameter('queue_size').value

    # Use the parameters
    self._pub = self.create_lifecycle_publisher(String, topic, qsize
   )
    self._timer_period = 1.0 / rate

    self.get_logger().info(
        f'Configured: topic={topic}, rate={rate} Hz, queue={qsize}')
    return TransitionCallbackReturn.SUCCESS
```

Listing 21: *Declaring parameters in on_configure (Python)*

```cpp
CallbackReturn LifecycleTalker::on_configure(
  const rclcpp_lifecycle::State & state)
{
```

```
  // Declare with defaults
  declare_parameter("publish_rate", 1.0);
  declare_parameter("topic_name", std::string("lifecycle_chatter"));
  declare_parameter("queue_size", 10);

  // Read
  auto rate  = get_parameter("publish_rate").as_double();
  auto topic = get_parameter("topic_name").as_string();
  auto qsize = static_cast<size_t>(
                get_parameter("queue_size").as_int());

  // Use
  pub_ = create_lifecycle_publisher<std_msgs::msg::String>(topic,
   qsize);
  timer_period_ = std::chrono::duration<double>(1.0 / rate);

  RCLCPP_INFO(get_logger(),
    "Configured: topic=%s, rate=%.1f Hz, queue=%zu",
    topic.c_str(), rate, qsize);
  return CallbackReturn::SUCCESS;
}
```

Listing 22: *Declaring parameters in on_configure (C++)*

7.2 Error Handling and Recovery

```
def on_configure(self, state: State) -> TransitionCallbackReturn:
    try:
        # Attempt to open hardware device
        self._device = open_hardware('/dev/sensor0')
        self._pub = self.create_lifecycle_publisher(
            SensorData, 'sensor_data', 10)
        return TransitionCallbackReturn.SUCCESS

    except DeviceNotFoundError as e:
        # FAILURE: node returns to previous primary state (
    Unconfigured)
        self.get_logger().error(f'Device not found: {e}. Retryable.'
    )
        return TransitionCallbackReturn.FAILURE

    except Exception as e:
        # ERROR: node calls on_error(); if that also fails ->
    Finalized
        self.get_logger().fatal(f'Unexpected error: {e}')
        return TransitionCallbackReturn.ERROR
```

Listing 23: *Returning FAILURE vs ERROR (Python)*

7.3 Using a Launch File with Lifecycle Nodes

```
# Copyright 2024 Developer
# SPDX-License-Identifier: Apache-2.0
```

```python
from launch import LaunchDescription
from launch_ros.actions import LifecycleNode
from launch_ros.actions import Node
from launch.actions import EmitEvent, RegisterEventHandler
from launch_ros.events.lifecycle import ChangeState
from launch_ros.event_handlers import OnStateTransition
from lifecycle_msgs.msg import Transition


def generate_launch_description():

    # Declare the talker as a LifecycleNode action
    talker = LifecycleNode(
        package='my_lifecycle_py',
        executable='lifecycle_talker',
        name='lifecycle_talker',
        namespace='',
        output='screen',
    )

    # Declare the listener as a LifecycleNode action
    listener = LifecycleNode(
        package='my_lifecycle_py',
        executable='lifecycle_listener',
        name='lifecycle_listener',
        namespace='',
        output='screen',
    )

    # Automatically configure the talker when it reaches '
    unconfigured'
    configure_talker = EmitEvent(
        event=ChangeState(
            lifecycle_node_matcher=lambda _: True,  # match all
            transition_id=Transition.TRANSITION_CONFIGURE,
        )
    )

    # When talker reaches 'inactive', activate it
    activate_on_inactive = RegisterEventHandler(
        OnStateTransition(
            target_lifecycle_node=talker,
            goal_state='inactive',
            entities=[
                EmitEvent(event=ChangeState(
                    lifecycle_node_matcher=lambda node: node ==
    talker,
                    transition_id=Transition.TRANSITION_ACTIVATE,
                ))
            ],
        )
    )

    return LaunchDescription([
```

```
        talker,
        listener,
        configure_talker,
        activate_on_inactive,
    ])
```

Listing 24: *launch/lifecycle_demo.launch.py*

```
ros2 launch my_lifecycle_py lifecycle_demo.launch.py
```

Listing 25: *Running the launch file*

7.4 Introspecting Lifecycle Services

Every lifecycle node exposes the following standard services:

Service	Type
`/<name>/get_state`	`lifecycle_msgs/srv/GetState`
`/<name>/change_state`	`lifecycle_msgs/srv/ChangeState`
`/<name>/get_available_states`	`lifecycle_msgs/srv/GetAvailableStates`
`/<name>/get_available_transitions`	`lifecycle_msgs/srv/GetAvailableTransitions`
`/<name>/get_transition_graph`	`lifecycle_msgs/srv/GetAvailableTransitions`

```
# Get current state
ros2 service call /lifecycle_talker/get_state \
  lifecycle_msgs/srv/GetState

# List available transitions from current state
ros2 service call /lifecycle_talker/get_available_transitions \
  lifecycle_msgs/srv/GetAvailableTransitions

# Trigger configure via service call
ros2 service call /lifecycle_talker/change_state \
  lifecycle_msgs/srv/ChangeState \
  "{transition: {id: 1, label: configure}}"
```

Listing 26: *Calling lifecycle services directly*

8 Best Practices

1. **Allocate in `on_configure`, start in `on_activate`.** Create publishers, subscribers, and service clients during configuration. Start timers and begin processing only on activation.
2. **Release symmetrically.** If you create something in `on_configure`, release it in `on_cleanup` (not `on_shutdown` only). Mirror every allocation with a corresponding teardown.
3. **Always call the parent implementation** for `on_activate` and `on_deactivate` when using lifecycle publishers; the parent toggles the publisher's activation flag.
4. **Use lifecycle publishers** (not regular publishers) for topics that should be silent in the Inactive state. Lifecycle publishers refuse to publish unless activated.

5. **Distinguish FAILURE from ERROR**. Return `FAILURE` for expected, recoverable problems (hardware not ready, parameter out of range). Return `ERROR` only for truly unexpected conditions.

6. **Keep callbacks fast**. Transition callbacks block the executor. Offload long-running initialisation (e.g. loading a large ML model) to a thread and return `SUCCESS` once the thread completes.

7. **Test each transition independently**. Write unit tests for each callback; do not rely solely on integration tests.

8. **Log state transitions**. Always log at the entry of each callback. Include the state label (`state.label()`) to make logs traceable.

9. **Avoid shared mutable state between states**. If a member variable is set in `on_configure` and used in `on_activate`, document this dependency clearly.

10. **Use a lifecycle manager for multi-node systems**. The **`nav2_lifecycle_manager`** package provides a production-grade manager that handles dependency ordering and error recovery.

Warning

Do **not** attempt to publish messages from a lifecycle publisher while the node is in the *Inactive* state. The message will be silently dropped. Always check `pub.is_activated()` (C++) or `pub.is_activated` (Python) before calling `publish()`.

9 Quick Reference Card

	Python	C++
Base class	`rclpy.lifecycle.LifecycleNode`	`rclcpp_lifecycle::LifecycleNode`
Return type	`TransitionCallbackReturn`	`CallbackReturn`
Lifecycle publisher	`create_lifecycle_publisher()`	`create_lifecycle_publisher<T>()`
Activate publisher	Call `super().on_activate(state)`	Call `LifecycleNode::on_activate(state)`
Check activated	`pub.is_activated`	`pub_->is_activated()`
Get state (CLI)	`ros2 lifecycle get /<node_name>`	
Set transition (CLI)	`ros2 lifecycle set /<node_name> <transition>`	
Monitor events	`ros2 topic echo /<node_name>/transition_event`	

A Installing ROS 2 and Setting Up a Development Workspace on Ubuntu

This appendix guides you through installing ROS 2 on Ubuntu 22.04 (for *Humble Hawksbill*, the current LTS) and preparing a colcon workspace.

A.1 Supported Ubuntu Versions

ROS 2 Distribution	Ubuntu	EOL
Humble Hawksbill (LTS)	22.04 (Jammy)	May 2027
Iron Irwini	22.04 / 23.04	Nov 2024
Jazzy Jalisco (LTS)	24.04 (Noble)	May 2029

The steps below use **Humble on Ubuntu 22.04**. Replace `humble` with `jazzy` (and `jammy` with `noble`) for the Jazzy release.

A.2 Step 1 -- Set the Locale

```
sudo apt update && sudo apt install -y locales
sudo locale-gen en_US en_US.UTF-8
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
export LANG=en_US.UTF-8
```

A.3 Step 2 -- Add the ROS 2 APT Repository

```
# Install prerequisite tools
sudo apt install -y \
  software-properties-common \
  curl \
  gnupg \
  lsb-release

# Add the ROS 2 GPG key
sudo curl -sSL \
  https://raw.githubusercontent.com/ros/rosdistro/master/ros.key \
  -o /usr/share/keyrings/ros-archive-keyring.gpg

# Add the repository to sources list
echo \
  "deb [arch=$(dpkg --print-architecture) \
  signed-by=/usr/share/keyrings/ros-archive-keyring.gpg] \
  https://packages.ros.org/ros2/ubuntu \
  $(. /etc/os-release && echo $UBUNTU_CODENAME) main" | \
  sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null

sudo apt update
```

A.4 Step 3 -- Install ROS 2

```
# Full desktop install (recommended -- includes RViz, demos,
   tutorials)
sudo apt install -y ros-humble-desktop

# OR minimal base install (headless / servers)
```

```
# sudo apt install -y ros-humble-ros-base

# Developer tools (colcon, rosdep, etc.)
sudo apt install -y \
  ros-dev-tools \
  python3-colcon-common-extensions \
  python3-rosdep \
  python3-argcomplete
```

A.5 Step 4 -- Initialise rosdep

```
sudo rosdep init
rosdep update
```

A.6 Step 5 -- Source the Installation

Add the following line to your `~/.bashrc` so ROS 2 is sourced in every new terminal:

```
echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc
source ~/.bashrc
```

Verify the installation:

```
ros2 --version
# Expected output: ros2 cli... version X.Y.Z
```

A.7 Step 6 -- Create and Initialise a Colcon Workspace

```
# Create the workspace directory structure
mkdir -p ~/ros2_ws/src
cd ~/ros2_ws

# Build an empty workspace to create install/ and build/ directories
colcon build

# Source the workspace overlay
echo "source ~/ros2_ws/install/setup.bash" >> ~/.bashrc
source ~/.bashrc
```

A.8 Step 7 -- Clone or Create Your Packages

```
# Enter the source directory
cd ~/ros2_ws/src

# Option A: clone an existing package
git clone https://github.com/ros2/demos.git

# Option B: create a new Python package from scratch
ros2 pkg create --build-type ament_python \
  --dependencies rclpy lifecycle_msgs std_msgs \
  my_lifecycle_py

# Option C: create a new C++ package from scratch
ros2 pkg create --build-type ament_cmake \
  --dependencies rclcpp rclcpp_lifecycle lifecycle_msgs std_msgs \
  my_lifecycle_cpp
```

A.9 Step 8 -- Install Package Dependencies

```
# From workspace root
cd ~/ros2_ws

# Install all dependencies listed in package.xml files
rosdep install --from-paths src --ignore-src -r -y
```

A.10 Step 9 -- Build and Test

```
# Full build
colcon build --symlink-install

# Build a specific package
colcon build --packages-select my_lifecycle_cpp

# Run tests
colcon test
colcon test-result --verbose
```

A.11 Step 10 -- Useful Development Tools

A.11.1 colcon Cheatsheet

```
# Build with verbose output
colcon build --event-handlers console_direct+

# Build in parallel (N jobs)
```

```
colcon build --parallel-workers 4

# Build only packages that have changed
colcon build --packages-above-and-dependencies my_lifecycle_cpp

# List all packages in workspace
colcon list

# Clean build artefacts (keep install/)
rm -rf build/ log/
```

A.11.2 ROS 2 Environment Utilities

```
# List all running nodes
ros2 node list

# Inspect a node's interfaces
ros2 node info /lifecycle_talker

# List all topics
ros2 topic list

# List all services
ros2 service list

# List all lifecycle nodes
ros2 lifecycle nodes

# Echo a topic with a rate limit
ros2 topic echo /lifecycle_chatter --max-count 5
```

A.11.3 Installing Extra Packages

```
# Lifecycle messages (included in desktop; explicit install for base
    )
sudo apt install -y ros-humble-lifecycle-msgs

# nav2 lifecycle manager (production-grade orchestration)
sudo apt install -y ros-humble-nav2-lifecycle-manager

# rqt plugins for graphical introspection
sudo apt install -y ros-humble-rqt-graph ros-humble-rqt-topic
```

A.11.4 Optional: Install Visual Studio Code with ROS Extension

```
# Install VS Code (if not already present)
sudo snap install code --classic
```

```
# Inside VS Code, install these extensions:
# - "ROS" by Microsoft     (ms-iot.vscode-ros)
# - "C/C++" by Microsoft  (ms-vscode.cpptools)
# - "Python" by Microsoft (ms-python.python)
```

A.12 Summary: Full Install Script

```
#!/usr/bin/env bash
set -e

# 1. Locale
sudo apt update && sudo apt install -y locales
sudo locale-gen en_US en_US.UTF-8
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
export LANG=en_US.UTF-8

# 2. Repository
sudo apt install -y software-properties-common curl gnupg lsb-
    release
sudo curl -sSL \
  https://raw.githubusercontent.com/ros/rosdistro/master/ros.key \
  -o /usr/share/keyrings/ros-archive-keyring.gpg
echo "deb [arch=$(dpkg --print-architecture) \
  signed-by=/usr/share/keyrings/ros-archive-keyring.gpg] \
  https://packages.ros.org/ros2/ubuntu \
  $(. /etc/os-release && echo $UBUNTU_CODENAME) main" | \
  sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null
sudo apt update

# 3. Install
sudo apt install -y ros-humble-desktop ros-dev-tools \
  python3-colcon-common-extensions \
  python3-rosdep python3-argcomplete

# 4. rosdep
sudo rosdep init && rosdep update

# 5. Source
echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc
source ~/.bashrc

# 6. Workspace
mkdir -p ~/ros2_ws/src
cd ~/ros2_ws && colcon build
echo "source ~/ros2_ws/install/setup.bash" >> ~/.bashrc
source ~/.bashrc

echo "ROS 2 Humble installed successfully!"
ros2 --version
```

Listing 27: *One-shot install script for ROS 2 Humble on Ubuntu 22.04*

Note

Run this script *once* on a fresh Ubuntu 22.04 installation. Do **not** run it inside a Docker container without adjusting the `sudo` invocations, and do **not** run as root (omit sudo and adjust paths as needed).

B Troubleshooting

Problem	Solution
`ros2: command not found`	Run `source /opt/ros/humble/setup.bash` or add it to `~/.bashrc`.
Node not found after build	Source the workspace overlay: `source ~/ros2_ws/install/setup.bash`.
Messages not received	Check that both talker and listener are in *Active* state. Use `ros2 lifecycle get` to inspect.
`change_state` service returns failure	The transition may not be valid from the current state. Use `ros2 lifecycle list` to see available transitions.
Publisher drops messages silently	The lifecycle publisher is inactive. Ensure you called `super().on_activate(state)` (Python) or `LifecycleNode::on_activate(state)` (C++).
Colcon build fails with missing dep	Run `rosdep install --from-paths src --ignore-src -r -y` from the workspace root.
Linking error in C++	Add `rclcpp_lifecycle` to both `find_package` and `ament_target_dependencies` in CMakeLists.txt.
Python `ImportError` on `rclpy.lifecycle`	Install the package: `sudo apt install ros-humble-rclpy`.